

Hierarchical Navigable Small World (HNSW) Graphs

Page 1 of 2

Hierarchical Navigable Small World (HNSW) is a state-of-the-art algorithm for Approximate Nearest Neighbor (ANN) search in high-dimensional vector spaces. In vector retrieval, calculating the exact Euclidean or Cosine distance between a query vector and millions of database vectors is computationally prohibitive. HNSW solves this by structuring the vector database as a multi-layer graph, achieving logarithmic search complexity $O(\log N)$ while maintaining high retrieval accuracy (recall).

The design of HNSW is inspired by the Skip List data structure and the Small World graph concept. A skip list is a linked list with hierarchical layers of shortcuts that allow fast search. Similarly, HNSW builds a multi-layered graph where the bottom layer (Layer 0) contains all vectors connected in a dense graph, and upper layers contain fewer vectors, forming coarser graphs with longer-range links. This hierarchical setup prevents search queries from getting stuck in local minima in high-dimensional space.

Hierarchical Navigable Small World (HNSW) Graphs

Page 2 of 2

The search and insertion processes in HNSW operate as follows:

1. Search: The query begins at an entry point in the top-most layer. The algorithm performs a greedy search, moving from node to neighbor, until it reaches a local minimum (a node where no neighbor is closer to the query vector). It then drops down to the corresponding node in the next layer and repeats the search. This continues until it reaches Layer 0, where it performs a local search to find the final K nearest neighbors.
2. Insertion: When a new vector is added, its maximum layer is determined randomly using an exponential decay distribution. The algorithm traverses the graph from the top layer down to the insertion layer to find the nearest entry points, then inserts the node at each layer and connects it to its M nearest neighbors. Parameter 'M' controls connection density, and 'ef_construct' controls the search queue size during build time, balancing build speed and recall.